5 Implementing consumer-driven contract testing for mobile clients

This chapter covers

- Pact consumer setup for Android apps
- Android consumer test implementation
- Android consumer contract generation

Chapter 4 covered consumer-driven contract testing (CDCT) for web clients using Pact (specification version V3). However, if you're a mobile developer or tester, you'll want to understand how to implement CDCT in your native language. In this chapter, we'll use Kotlin and Gradle to walk through the implementation steps of CDCT in an Android-native application using Pact (specification version V4), from setting up the interaction and applying best practices to generating the consumer contract. Skip to chapter 6 if mobile isn't something you need at the moment.

If you have a strong understanding of the foundational concepts from chapter 3, it's time to start implementing contract testing. In our experience of working with mobile applications within a microservices environment, it's important to take care of external APIs interacting with the mobile app. Mobile apps are released in a bundled version at a point in time, but not every user updates the app at the same time. Good coverage around the contracts is key to providing a seamless experience for the user.

Imagine you're developing the mobile app for the new search feature, returning a list of movies. The API specification defines the `'/movies'` endpoint with an example response containing the ID, name, and release date. The API and mobile app are built based on the specification. Imagine the mobile app implementation changes the date field from a date type to a string in their tests that mock out the API service. They realize they only need to display the year of the movie's release, so there is no need to consume the raw date. However, due to time pressures, the API team forgot